Random Numbers

- The .NET Framework provides the Random class that you can use in C# to generate random numbers.
- First you create an object from the Random class with a statement such as this: Random rand = new Random();
- The first part of the statement, appearing on the left side of the = operator, declares a variable named rand that can be used to reference a Random object:

```
Random rand = new Random();
This declares a variable named rand
that can reference a Random object.
```

- The second part of the statement, appearing on the right side of the = operator, causes an object of the Random class to be created in memory:

```
Random rand = new Random();
This creates a Random object
in memory.
```

- The = operator causes the rand variable to reference the Random object.

Next Method

- Once you have created a Random object, you can call its Next method to get a random integer number.
- Example:

```
// Declare an int variable.

int number;

// Create a Random object.

Random rand = new Random();

// Get a random integer and assign it to number.
number = rand.Next();
```

- After this code executes, the number variable contains a random integer.
- If you call the Next method with no arguments, as shown in this example, the returned integer is somewhere between 0 and 2,147,483,647.

Next Method

- you can pass an argument that specifies an upper limit to the generated number's range.
- In the following statement, the value assigned to number is somewhere between 0 and 99:

```
number = rand.Next(100);
```

- You can add or subtract a value to shift the numeric range upward or downward. In the following statement, we call the Next method to get a random number in the range of 0 through 9, and then we add 1 to it.
- So, the number assigned to number is somewhere in the range of 1 through 10:

```
number = rand.Next(10) + 1;
```

NextDouble Method

- You can call a Random object's NextDouble method to get a random floating-point number between 0.0 and 1.0 (not including 1.0).
- Example:

```
// Declare a Double variable.

double number;

// Create a Random object.

Random rand = new Random();

// Get a random number and assign it to number.
number = rand.NextDouble();
```

Example 13

Simulating Coin tosses

Create an application that simulates the tossing of a coin. Each time the user tosses the coin, the application uses a Random object to get a random integer in the range of 0 through 1. If the random number is 0, it means the tails side of the coin is up, and if the random number is 1, it means the heads side is up. The application displays an image of a coin showing either heads or tails, depending on the value of the random number.

Example 13- Control property settings

```
Control Name  Control Type _ Property Settings

headsPictureBox PictureBox  Image: Select and import the Heads1.bmp file
from the Chap05 folder of the Student Sample
Programs.
Size: 170, 170
SizeMode: Zoom
Visible: True

tailsPictureBox PictureBox  Image: Select and import the Tails1.bmp file
from the Chap05 folder of the Student Sample
Programs.
Size: 170, 170
SizeMode: Zoom
Visible: False

tossButton  Button  Text: Toss

exitButton  Button  Text: Exit
```

Example 13- Code

```
private void tossButton_Click(object sender, EventArgs e)
{
// Variable to indicate which side is up
int sideUp;
// Create a Random object.
Random rand = new Random();
// Get a random integer in the range of 0 through 1.
// 0 means tails up, 1 means heads up.
sideUp = rand.Next (2);
// Display the side that is up.
if (sideUp == 0)
{
// Display tails up.
tailsPictureBox.Visible = true;
headsPictureBox.Visible = false;
}
else
{
// Display heads up.
headsPictureBox.Visible = true;
tailsPictureBox.Visible = false;
}
}
private void exitButton_Click(object sender, EventArgs e)
{
// Close the form.
this.Close();
}
```

Load Event

- When an application's form loads into memory, an event known as the Load event takes place.
- You can write an event handler for the Load event, and that handler will execute just before the form is displayed.
- To create a Load event handler for a form, simply double-click any area of the form in the Designer window where there is no other control.
- The code editor will open with an empty Load event handler. If the form is named Form1, the event handler is named Form1_Load.
- Any code that you write inside the event handler executes when the form's Load event takes place.
- Example:

```
private void Forml_Load(object sender, EventArgs e)
{

MessageBox.Show("Prepare to see the form!");
}
```


THANK YOU

Chapter 5

Modularizing Your Code with Methods

By

Dr. Sara Tarik

Introduction to Methods

- Methods can be used to break a complex program into small, manageable pieces.
- A void method simply executes a group of statements and then terminates.
- A value-returning method returns a value to the statement that called it.
- A method is a collection of statements that performs a specific task.
- So far, you have experienced methods in the following two ways:
  - You have created event handlers. An event handler is a special type of method that responds to events.
  - You have executed predefined methods from the .NET Framework, such as MessageBox.Show and the TryParse methods.
- In this chapter, you will learn how to create your own methods that can be executed just as you execute the .NET Framework methods.

Introduction to Methods

- The comparison between two programs: one that uses a long, complex event handler containing all the statements necessary to solve a problem as shown in left and another that divides a problem into smaller problems, each of which are handled by a separate method as shown in right.

```
namespace Example
{
public partial class Form1 : Form
{
private void myButton_Click(object sender, EventArgs e)
{
statement;
statement;
statement;
statement;
statement;  In this program the task is performed
statement;  by one long seqeunce of statements in
statement;  an event handler.
statement;
statement;
statement;
statement;
statement;
statement;
.. and so on.
}
}
}
```

```
namespace Example
{
public partial class Form1 : Form
{
private void myButton_Click(object sender, EventArgs e)
{
Method? () -
Method3() 7  CH
Method4();  Handler
}
private void Method2()
{
Star omeuty  In this program the task has
oe ee  Method 2  been divided into smaller tasks,
}  .                    each of which is performed
by a separate method.
private void Method3()
{
statement;
statement; Method 3
statement;
}
private void Method4()
{
statement;
statement; Method 4
statement;
}
}
}
```

Introduction to Methods

- In general terms, a program that is broken into smaller units of code, such as methods, is known as a modularized program.
- Modularization tends to simplify code.
- If a specific task is performed in several places in a program, a method can be written once to perform that task and then be executed any time it is needed.
- This benefit of using methods is known as code reuse because you are writing the code to perform a task once and then reusing it each time you need to perform the task.

void Methods

- A void method performs a task and then terminates.
- It does not return a value back to the statement that called it.
- A method definition has two parts: a header and a body.
  - The method header, which appears at the beginning of a method definition, lists several important things about the method, including the method's name.
  - The method body is a collection of statements that are performed when the method is executed.

Method Header

- Parts of method header:

```
Access  Return  Method
modifier  type  name  Parentheses
private void DisplayMessage()
{
MessageBox.Show("This is the DisplayMessage method.");
}
```

- Access modifier: The keyword private is an access modifier. When a method is declared as private, it can be called only by code inside the same class as the method. Alternatively, a method that is declared as public can be called by code that is outside the class. This is important because some applications have multiple classes, and unless you specifically intend a method to be available to code outside the class, you should declare it private.

Method Header

- Return type: When the keyword void appears here, it means that the method is a void method, and does not return a value. As you will see later in this chapter, a value-returning method lists a data type here.
- Method name: You should give each method a descriptive name. In general, the same rules that apply to variable names also apply to method names.
  - we use Pascal case for method names. Pascal case is like camelCase (the convention we have been using for variable names), except in a Pascal case name the first character is always uppercase.
- Parentheses: In the header, the method name is always followed by a set of parentheses

The method header is never terminated with a semicolon.

Method Body

- Beginning at the line after the method header, one or more statements appear inside a set of curly braces ({ }).
- These statements are the method's body and are performed any time the module is executed.
- When you write a method definition, Visual Studio automatically indents the statements in the method body.
- The indentation is not required, but it makes the code easier to read and debug.

Declaring Methods Inside a Class

- Methods usually belong to a class, so you must write a method's definition inside the class to which it is supposed to belong.
- In this chapter, all the methods that you will write will belong to an application's Form1 class.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Ling;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace Example
{
public partial class Form1 : Form
{
public Form1()
{
InitializeComponent();
}
Your method definitions will apear
}  here, inside the Form! class.
}
```

Calling a Method

- When a method is called, the program branches to that method and executes the statements in its body.
- **Example:**

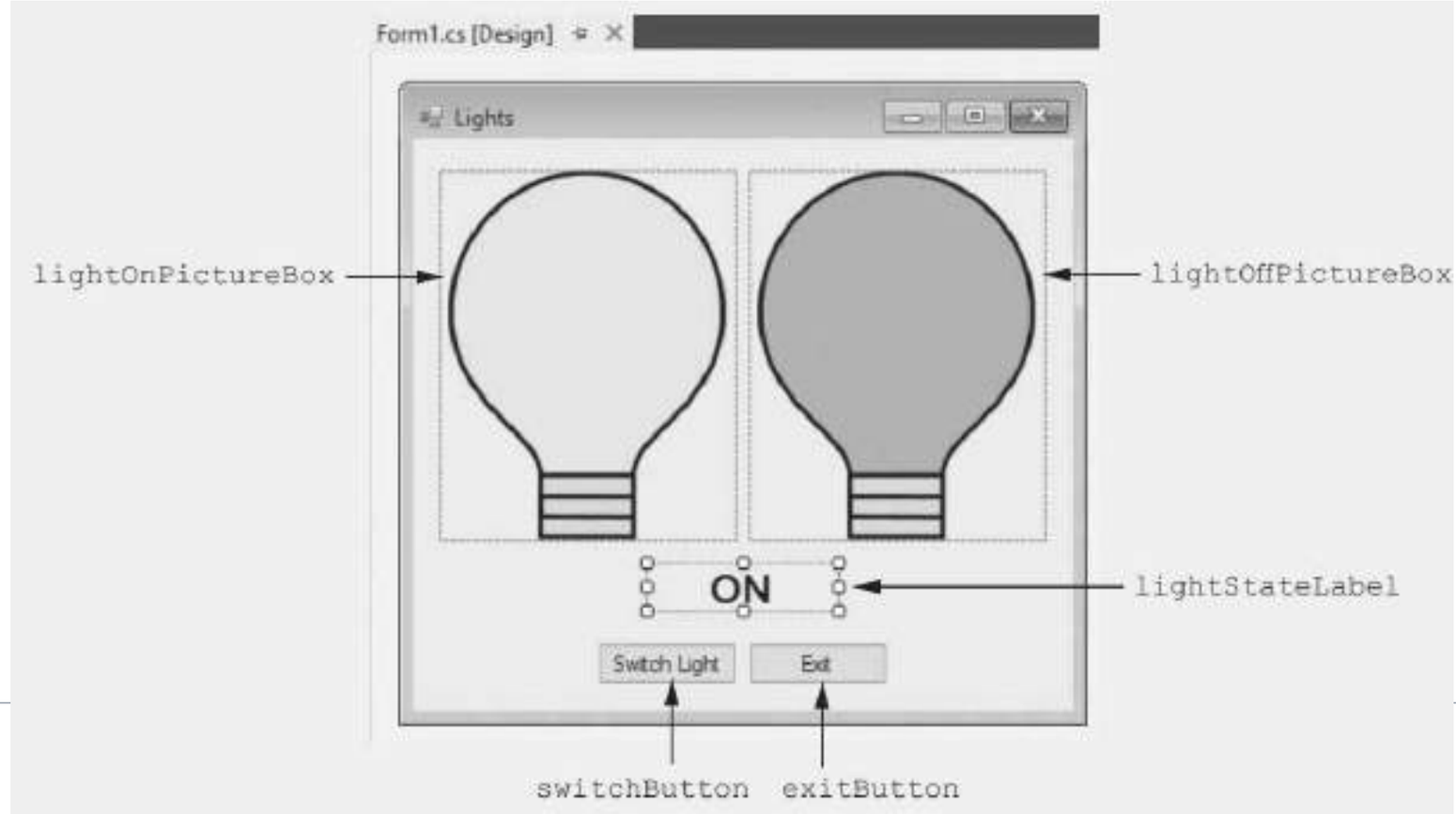
```
DisplayMessage();
```

- The statement is simply the name of the method followed by a set of parentheses. Because it is a complete statement, it is terminated with a semicolon.

Example 1

Lights application

When the user clicks the Switch Light button, the state of the light is reversed. In other words, if the light is currently on, it will be turned off. If the light is currently off, it will be turned on.



Example 1-property settings

Here are some details about specific property settings:

- The `lightOnPictureBox` control's Visible property is initially set to true.
- The `lightOffPictureBox` control's Visible property is initially set to false.
- The `lightStateLabel` displays either *ON* or *OFF* while the application runs to indicate whether the light is on or off. Initially, this control's Text property is set to *ON*.

When the light is turned on, the following actions take place:

- The `lightOnPictureBox` control's Visible property is set to `true`.
- The `lightOffPictureBox` control's Visible property is set to `false`.
- The `lightStateLabel` label's Text property is assigned the string `"ON"`.

When the light is turned off, the following actions take place:

- The `lightOffPictureBox` control's Visible property is set to `true`.
- The `lightOnPictureBox` control's Visible property is set to `false`.
- The `lightStateLabel` label's Text property is assigned the string `"OFF"`.

Example 1- Code

```
private void TurnLightOn()
{
// Display the "light on" image.
lightOnPictureBox.Visible = true;
// Hide the "light off" image.
lightOffPictureBox.Visible = false;
// Display the light's state.
lightStateLabel.Text = "ON";
}
private void TurnLightOff()
{
// Display the "light off" image.
lightOffPictureBox.Visible = true;
// Hide the "light on" image.
lightOnPictureBox.Visible = false;
// Display the light's state.
lightStateLabel.Text = "OFF";
}
private void switchButton_Click(object sender,
EventArgs e)
{
// Reverse the state of the light.
if (lightOnPictureBox.Visible == true)
{
TurnLightOff();
}
else
{
TurnLightOn();
}
}
private void exitButton_Click(object sender, EventArgs
e)
{
// Close the form,
this.Close();
}
```

Passing Arguments to Methods

- An argument is any piece of data that is passed into a method when the method is called.

```
MessageBox.Show("Hello");
```

  - This statement calls the MessageBox.Show method and passes the string "Hello" as an argument.

```
number = int.Parse(str);
```

  - Assume that number is an int variable and str is a string variable. This statement calls the int.Parse method, passing the str variable as an argument.

Passing Arguments to Methods

- A parameter is a variable that receives an argument that is passed into a method.
- If you are writing a method and you want it to receive arguments when it is called, you must equip the method with one or more parameter variables.

```
private void DisplayValue(int value)
{
MessageBox.Show(value.ToString());
}
```

- Notice the int variable declaration that appears inside the parentheses (int value). This is the declaration of a parameter variable, which enables the DisplayValue method to accept an int value as an argument.
- Here is an example of a call to the DisplayValue method, passing 5 as an argument:

```
DisplayValue(5);
```

Passing Arguments to Methods

- You may also pass the contents of variables and the values of expressions as arguments.
- For example, the following statements call the DisplayValue method with various arguments passed:

```
DisplayValue(x);
```

- It passes the value of the variable x as the argument to the DisplayValue method

```
DisplayValue(x * 4);
```

- It passes the result of the expression x * 4 as the argument to the DisplayValue method

```
DisplayValue(int.Parse("700"));
```

- It passes the value returned from the int.Parse method as the argument to the DisplayValue method. (The int.Parse method is called first, and its return value is passed to the DisplayValue method.)

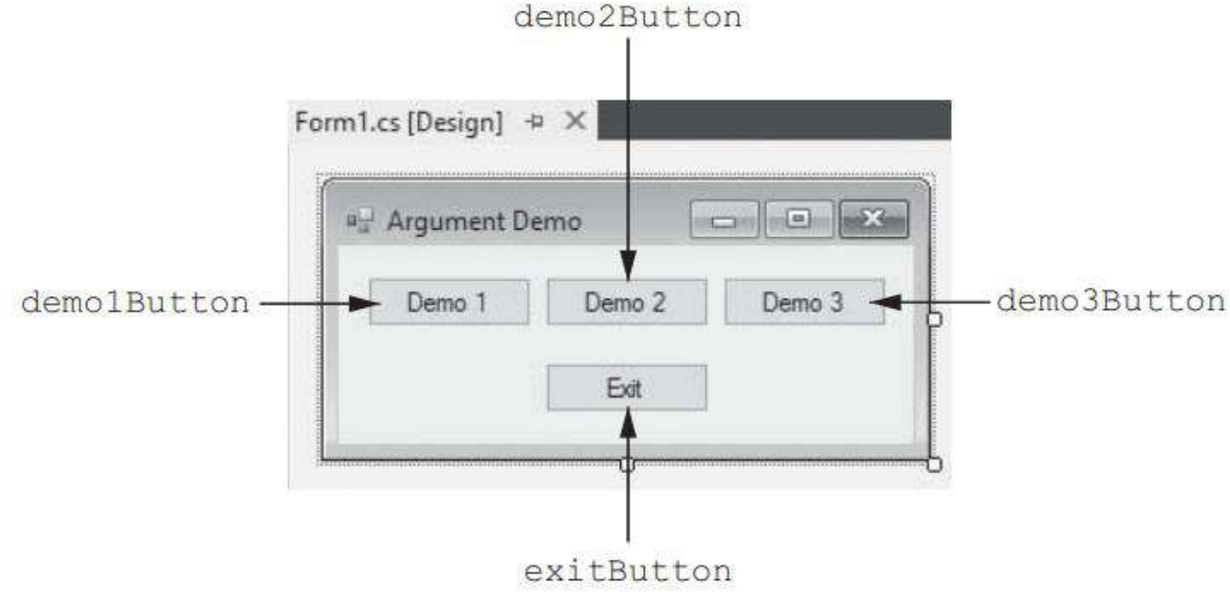
Example 2

Argument Demo application

If you run the application and click the Demo 1 button a message box showing the value 5.

If you click the Demo 2 button, a message box showing the value 8 (3 + 5).

If you click the Demo 3 button, a message box appear five times, showing the values 0 through 4.



Example 2- code

```
private void DisplayValue(int value)
{
MessageBox.Show(value.ToString());
}
private void demolButton_Click(object sender, EventArgs e)
{
// Call DisplayValue passing 5 as an argument.
DisplayValue (5);
}
private void demo2Button_Click(object sender, EventArgs e)
{
// Call DisplayValue passing the expression 3 + 5
// as an argument.
DisplayValue(3 + 5);
}
private void demo3Button_Click(object sender, EventArgs e)
{
// Use a loop to call DisplayValue 5 times.
for (int count = 0; count < 5; count++)
{
DisplayValue (count);
}
}
```

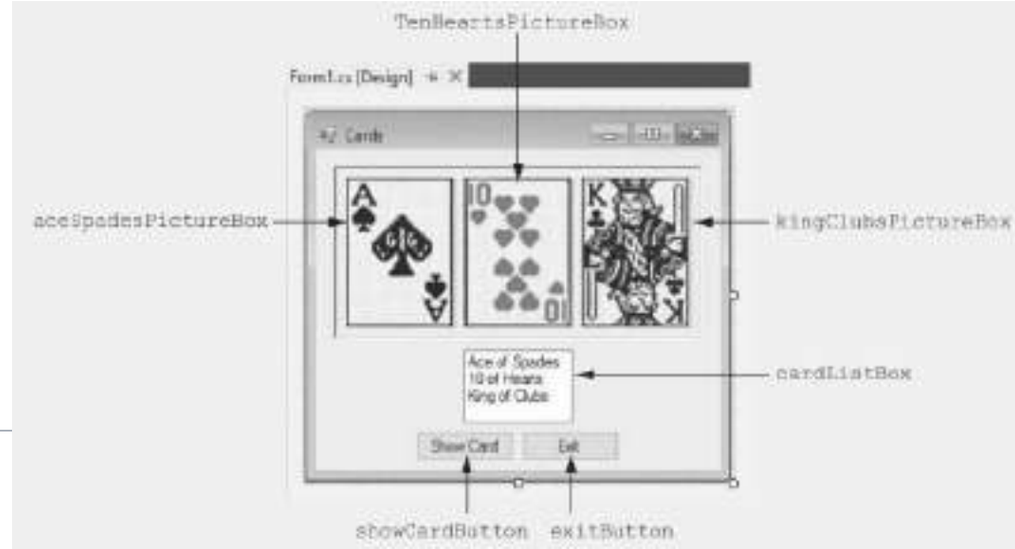
Example 3

Cards application

The PictureBox controls show the images of three cards.

Each PictureBox control's Visible property is set to False, so they do not initially appear when the application runs.

The user can select a card's name from the ListBox, click the Show Card button, and the image of the selected card will appear.



Example 3- Code

```
public Form1()
{
InitializeComponent();
// The ShowCard method accepts a string that names
// the selected card, and displays that card.
private void ShowCard(string card)
{
switch (card)
{
case "Ace of Spades" :
ShowAceSpades();
break;
case "10 of Hearts":
ShowTenHearts();
break;
case "King of Clubs":
ShowKingClubs();
break;
}
}
// The ShowAceSpades method makes the Ace of Spades
// visible and the other cards invisible.
private void ShowAceSpades()
{
aceSpadesPictureBox.Visible = true;
tenHeartsPictureBox.Visible = false;
kingClubsPictureBox.Visible = false;
}
// The ShowTenHearts method makes the Ten of Hearts
// visible and the other cards invisible.
private void ShowTenHearts()
{
tenHeartsPictureBox.Visible = true;
aceSpadesPictureBox.Visible = false;
kingClubsPictureBox.Visible = false;
}
// The ShowKingClubs method makes the King of Clubs
// visible and the other cards invisible.
private void ShowKingClubs()
{
kingClubsPictureBox.Visible = true;
aceSpadesPictureBox.Visible = false;
tenHeartsPictureBox.Visible = false;
}
```

```
private void showCardButton_Click(object sender, EventArgs e)
{
// If a card is selected in the ListBox, display it.
if (cardListBox.SelectedIndex != -1)
{
ShowCard (cardListBox.SelectedItem.ToString() );
}
else
{
MessageBox.Show("Please select a card from " +
"the list box.");
}
}
```

Passing Multiple Arguments

- The following code sample shows a method that accepts two arguments.

```
private void ShowMax(int numl, int num2)
2
 if (numl == num2)
 {
 MessageBox.Show("The numbers are equal.");
 }
 else if (numl > num2)
 {
)  MessageBox.Show(numl + " is the greatest.");
 }
 else
 {
 MessageBox.Show(num2 + " is the greatest.");
```

- Here is an example of a statement that calls the method:

```
ShowMax(5, 10);
```

- The arguments are passed into the parameter variables according to their positions.

Passing Multiple Arguments

- You have to write the data type for each parameter variable that is declared in a parameter list.
- For example, a compiler error would occur if the parameter list for the ShowMax method were written as shown here:

```
private void ShowMax(int num1, num2)      // Error!
```

- A data type for both the num1 and num2 parameter variables must be listed, as shown here:

```
private void ShowMax(int num1, int num2)
```

Named Arguments

- In addition to the conventional approach of positional argument passing (where the first argument is passed into the method's first parameter, the second argument is passed into the method's second parameter, and so forth).
- C# also allows you to specify which parameter an argument should be passed into:

parameterName : value

- In this format parameterName is the name of a parameter variable and value is the value being passed to that parameter.

Named Arguments

```
private void ShowName(string firstName, string lastName)
{

MessageBox.Show(firstName + " " + lastName);
}
```

- The following statement shows how the method can be called using named arguments:

ShowName(lastName : "Smith", firstName : "Suzanne");

  - This statement specifies that "Smith" should be passed into the lastName parameter and "Suzanne" should be passed into the firstName parameter.

- You get the same results as if you had called the method like this, using positional arguments:

ShowName("Suzanne", "Smith");

Default Arguments

- C# allows you to provide a default argument for a method parameter.
- When a default argument is provided for a parameter, it becomes possible to call the method without explicitly passing an argument into the parameter.
- Example:

```
private void ShowTax(decimal price, decimal taxRate = 0.07m)
{
// Calculate the tax.
decimal tax = price * taxRate;
// Display the tax.
MessageBox.Show("The tax is " + tax.ToString("c"));
}
```

- In this method definition, a default argument is provided for the taxRate parameter.
- Notice that the parameter name is followed by an equal sign and a value.
- The value that follows the equal sign is the default argument. In this case, the value is 0.07m is the default argument for the taxRate parameter. Because the taxRate parameter has a default argument, we have the option of omitting an argument for it when we call the method.

- Example: ShowTax(100.0m);

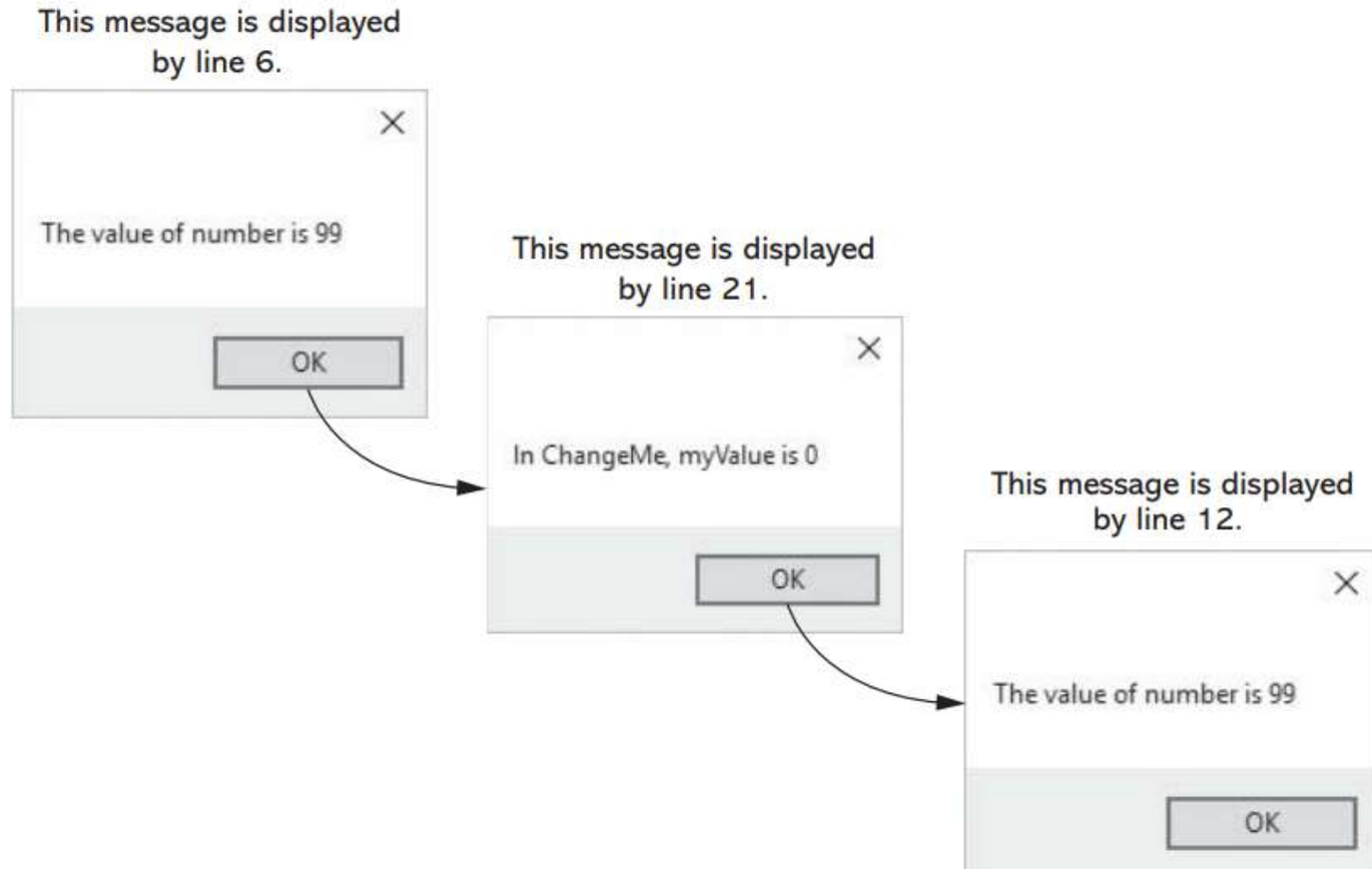
Passing Arguments by Value

- When an argument is passed by value, only a copy of the argument's value is passed into the parameter variable.
- If the contents of the parameter variable are changed inside the method, it has no effect on the argument in the calling part of the program.

Example 4

Pass By Value application

When you run the application and click the Go button, you see the sequence of message boxes shown in Figure



Example 4- Code

```
private void goButton_Click(object sender, EventArgs e)
{
 int number = 99;
 // Display the value of number.
 MessageBox.Show("The value of number is " + number);
 // Call ChangeMe, passing number as an argument.
 ChangeMe (number);
 // Display the value of number again.
 MessageBox.Show("The value of number is " + number);
}
private void ChangeMe(int myValue)
{
%  // Change the value of the myValue parameter.
 myValue = 0;
 // Display the value of myValue.
 MessageBox.Show("In ChangeMe, myValue is " + myValue);
}
```

Passing Arguments by Reference

- When an argument is passed by reference to a method, the method can change the value of the argument in the calling part of the program.
- When you want a method to be able to change the value of a variable that is passed to it as an argument, the variable must be passed by reference.
- In C#, there are two ways to pass an argument by reference:
  - You can use a reference parameter in the method.
  - You can use an output parameter in the method.

Reference Parameters

- When you pass an argument into a reference parameter, the reference parameter does not receive a copy of the argument's value.
- Instead, it becomes a reference to the argument that was passed into it.
- In C#, you declare a reference parameter by writing the ref keyword before the parameter variable's data type.
- **Example:**

```
private void SetToZero(ref int number)
{

number = 0;
}
```

Reference Parameters

```
int myVar = 99;
SetToZero(ref myVar);
```

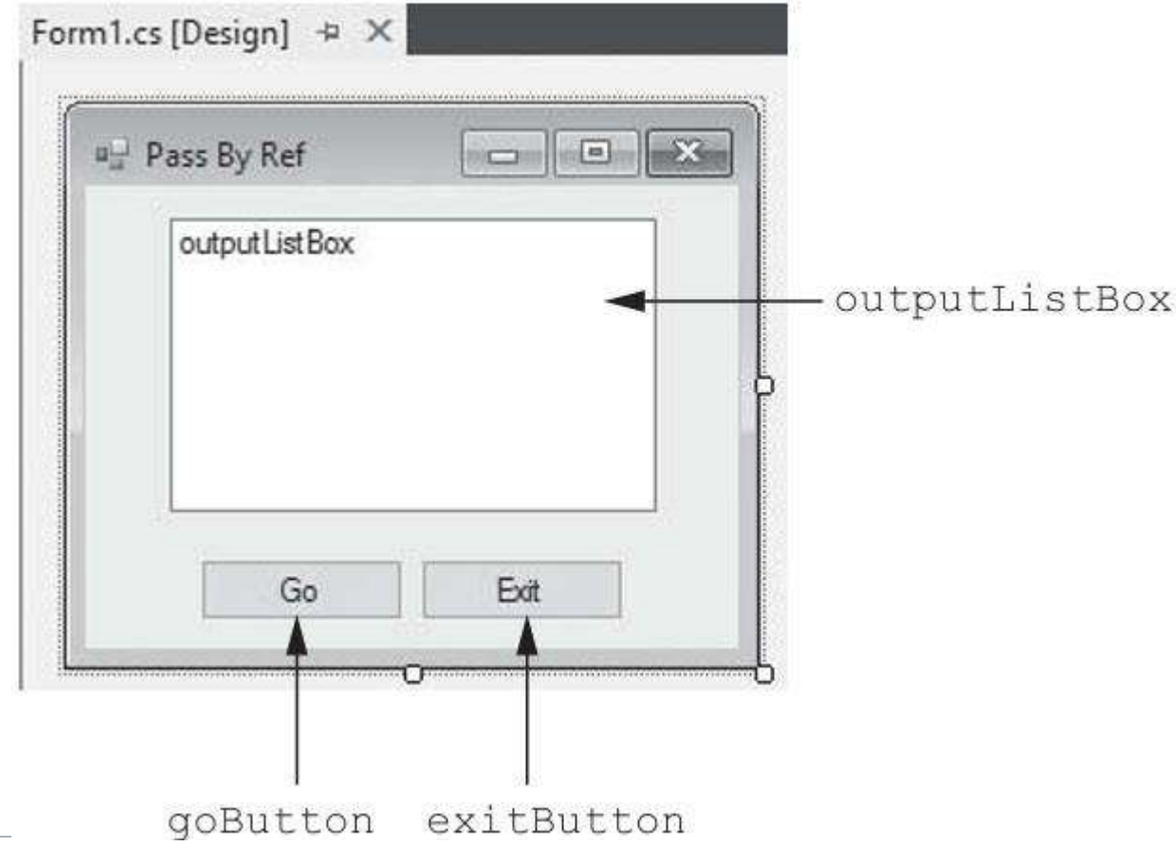
- The first statement declares myVar as an int variable, initialized with the value 99. The second statement calls the SetToZero method, passing myVar by reference. After the method call, the myVar variable is set to the value 0.
- When you pass an argument to a ref parameter, that argument must already be set to some value.
- For example, if a variable has not been initialized or assigned a value, you cannot pass it as an argument into a ref parameter.
- The following code sample causes a compiler error:

```
int myVar;  // Declare myVar with no initial value.
SetToZero(ref myVar); // Error! myVar is not set to a value.
```

Example 5

Pass By Ref application

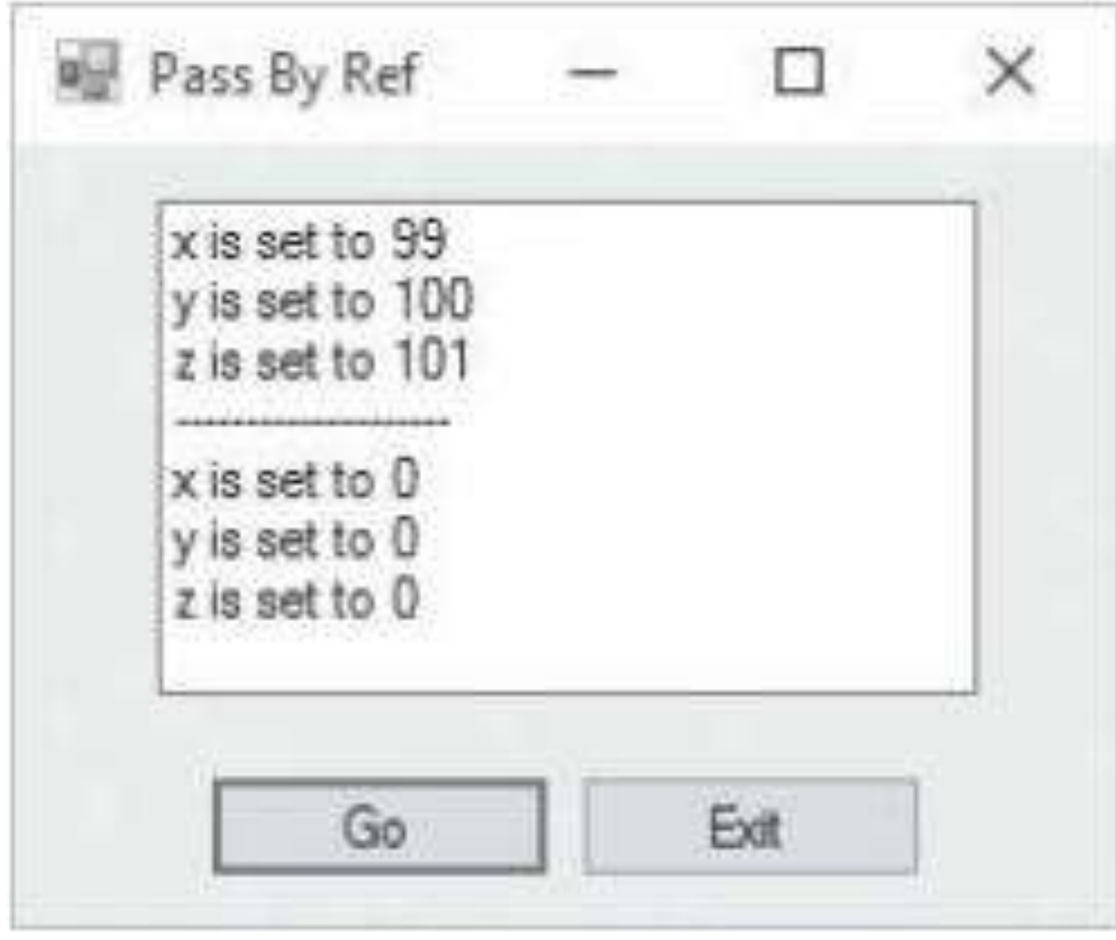
The Figure shows the application's form



Example 5- Code

```
public Form1()
{
InitializeComponent();
}
// The SetToZero method accepts an int argument
// by reference and sets it to zero.
private void SetToZero(ref int number)
{
number = 0;
}
private void goButton_Click(object sender, EventArgs e)
{
// Declare some local int variables.
int x = 99, y = 100, z = 101;
// Display the values in those variables.
outputListBox.Items.Clear();
outputListBox.Items.Add("x is set to " + x);
outputListBox.Items.Add("y is set to " + y);
outputListBox.Items.Add("z is set to " + z);
// Pass each variable to SetToZero.
SetToZero(ref x);
SetToZero(ref y);
SetToZero(ref z);
// Display the values in those variables again.
outputListBox.Items.Add("--------------------");
outputListBox.Items.Add("x is set to " + x);
outputListBox.Items.Add("y is set to " + y)7
outputListBox.Items.Add("z is set to "+ z);7
}
private void exitButton_Click(object sender, EventArgs e)
{
// Close the form.
this.Close();
}
```

Example 5- Pass By Ref application's output



Output Parameters

- An output parameter works like a reference parameter.
- Output parameters are different from reference parameters in the following ways:
  - An argument does not have to be set to a value before it is passed into an output parameter. For example, an uninitialized variable can be passed into an output parameter.
  - A method that has an output parameter must set the output parameter to some value before it finishes executing.
- In C#, you declare an output parameter by writing the out keyword before the parameter variable's data type.
- Example:

```
private void SetToZero(out int number)
{

number = 0;
}
```

Output Parameters

- When you call a method that has an output parameter, you must also write the keyword out before the argument.
- Example:

```
int myVar;
SetToZero (out myVar);
```

- The first statement declares myVar as an uninitialized int variable. The second statement calls the SetToZero method, passing myVar into the output parameter.
- After the method call, the myVar variable is set to the value 0.

Example 6

North America application

When the application runs and the user clicks the Get Countries button, the application uses an OpenFileDialog control to let the user select a file. The application reads each country name from the file and adds each one to the countriesListBox control.



Example 6- Code

```
// The GetFileName method gets a filename from the
// user and assigns it to the variable passed as
// an argument.
private void GetFileName(out string selectedFile)
{
if (openFile.ShowDialog() == DialogResult.OK)
{
selectedFile = openFile.FileName;
}
else
{
selectedFile = "";
}
}
```

Example 6- Code

```
// The GetCountries method accpets a filename as an
// argument.It opens the specified file and displays
// its contents in the countriesListBox control.
private void GetCountries(string filename)
{
try
{
// Declare a variable to hold a country name.
string countryName;
// Declare a StreamReader variable.
StreamReader inputFile;
// Open the file and get a StreamReader object.
inputFile = File.OpenText(filename);
// Clear anything currently in the ListBox.
countriesListBox.Items.Clear();
// Read the file's contents.
while (!inputFile.EndOfStream)
{
// Get a country name.
countryName = inputFile.ReadLine();
// Add the country name to the ListBox.
countriesListBox.Items.Add(countryName);
}
inputFile.Close();
}
catch (Exception ex)
{
// Display an error message.
MessageBox.Show(ex.Message);
}
}
```

Example 6- Code

```
private void getCountriesButton_Click(object sender, EventArgs e)
{

string filename;  // To hold the filename

// Get the filename from the user.

GetFileName (out filename);

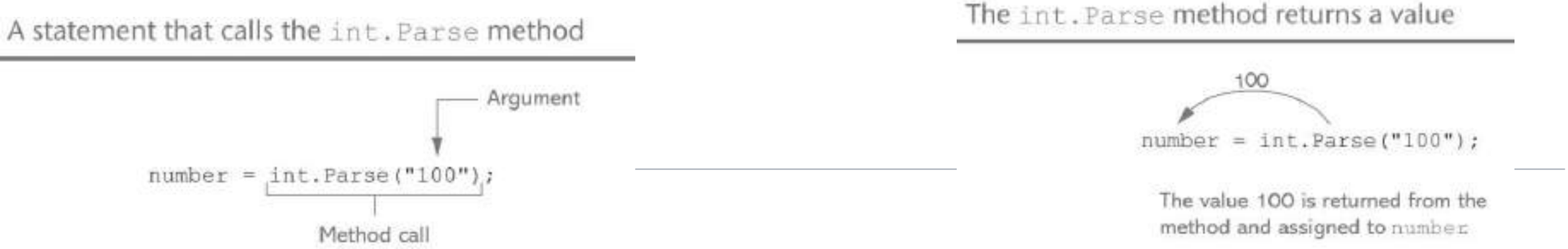
// Get the countries from the file.

GetCountries (filename);
}
```

Value-Returning Methods

- A value-returning method is a method that returns a value to the part of the program that called it.
- When a value-returning method finishes, however, it returns a value to the statement that called it.
- The value that is returned from a method can be used like any other value: It can be assigned to a variable, displayed on the screen, used in a mathematical expression (if it is a number), and so forth.
- For example, the int.Parse method accepts a string as an argument and returns the value of the string converted to an int.
- Example:

```
number = int.Parse("100");
```



Writing Your Own Value-Returning Methods

- You write a value-returning method in the same way that you write a void method, with two exceptions:
  - You must specify a data type for a value-returning method. The value that is returned from the method must be of the specified data type.
  - A value-returning method must have a return statement. The return statement causes a value to be returned from the method

Writing Your Own Value-Returning Methods

```
AccessModifier DataType MethodName(ParameterList)
{

statement;

statement;

etc.

return expression;
}
```

- AccessModifier: is an access modifier such as private or public.
- DataType: is the data type of the value that the method returns. We commonly call this the method's return type. For example, if the method returns an integer, the word int appears here. If the method returns a double value, then the word double appears here. Likewise, if the method returns a decimal value, the word decimal appears here.
- MethodName: is the name of the method.
- ParameterList: is an optional parameter list. If the method does not accept arguments, then an empty set of parentheses appears.

Writing Your Own Value-Returning Methods

- Parts of method header:

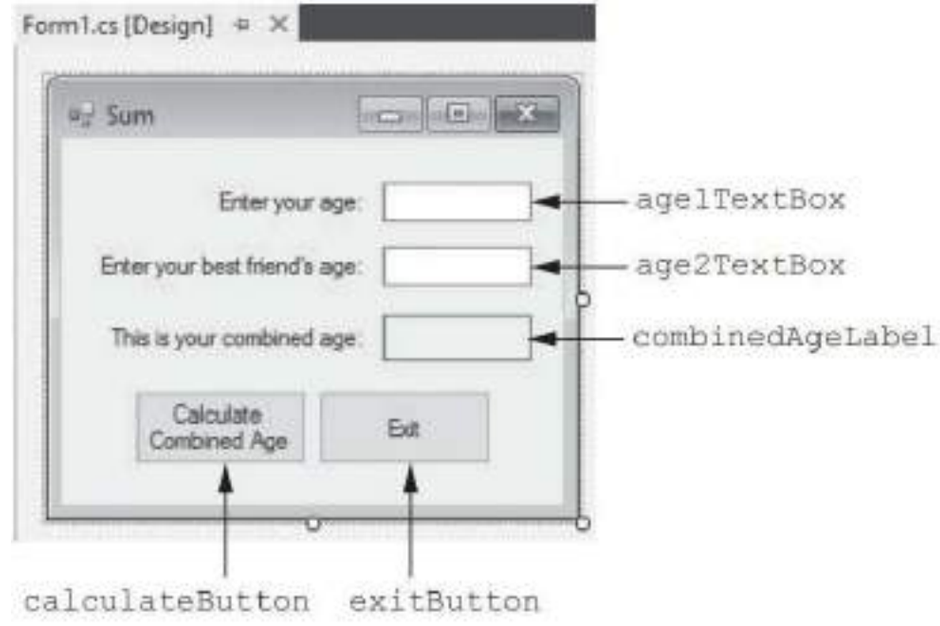
```
This is a private  The method  The method's  numlandnum2
method  returns anint  name is Sum  are parameters
private int Sum(int numl, int num2)
{
return numl + num2;
}
```

- The purpose of this method is to accept two int values as arguments and return their sum.
- Notice that the return statement returns the value of the expression num1 + num2.
- When the return statement executes, the method ends its execution and sends the value of num1 + num2 back to the part of the program that called the method.

Example 7

Sum application

When you run the application, you enter two integers into the age1TextBox and age2TextBox controls. When you click the Calculate Combined Age button, the sum of the two integers is displayed in the combinedAgeLabel control.



Example 7- Code

```
public Form1()
{
InitializeComponent();
}
// The Sum method accepts two int arguments
// and returns the sum of the arguments.
private int Sum(int num1, int num2)
{
return numl + num2;
}
private void calculateButton_Click(object sender, EventArgs e)
{
// Declare variables to hold two ages and their sum,
int userAge, friendAge, combinedAge;
// Get the user's age.
if (int.TryParse(agelTextBox.Text, out userAge))
{
// Get the best friend's age age.
if (int.TryParse(age2TextBox.Text, out friendAge) )
{
// Get the sum of the ages.
combinedAge = Sum(userAge, friendAge);
// Display the combined age.
combinedAgeLabel.Text = combinedAge.ToString();
1
else
{
// Display an error message.
MessageBox.Show("Enter an integer for your age.");
}
}
else
{
// Display an error message.
MessageBox.Show("Enter an integer for your age.");
}
}
```

Boolean Methods

- A Boolean method returns either true or false.
- You can use a Boolean method to test a condition, and then return either true or false to indicate whether the condition exists.
- For example, suppose you are writing a program that will ask the user to enter a number and then determine whether that number is even or odd.
  - Assume number is an int variable containing the number entered by the user.

```
if (number % 2 == 0)
{

MessageBox.Show("The number is even.");
I
else
{

MessageBox.Show("The number is odd.");
}
```

Boolean Method- Example

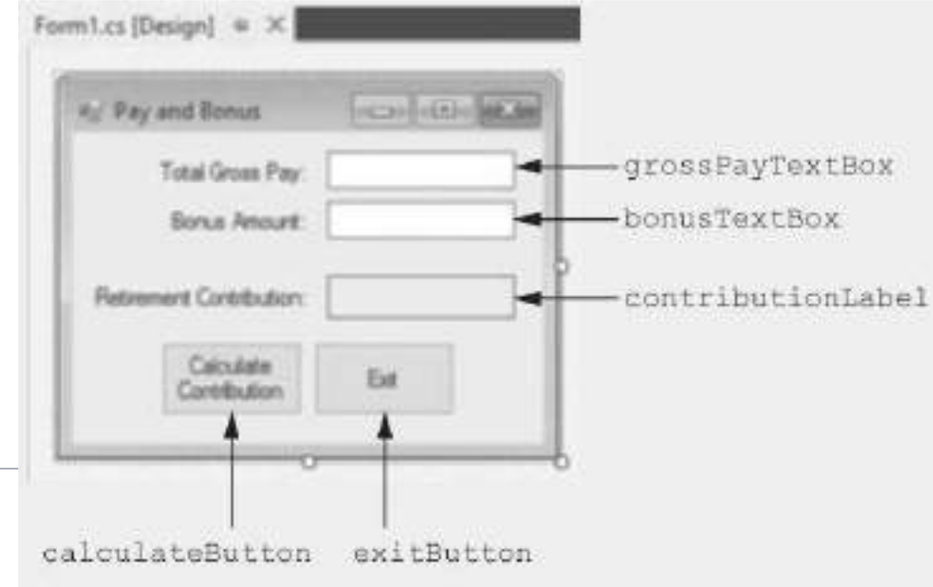
```
private bool IsEven(int number)
{
// Local variable to hold true or false
bool numberlIsEven;
// Determine whether the number is even.
if (number % 2 == 0)
{
numberIsEven = true;
}
else
{
numberIsEven = false;
}
// Return the result.
return numberIsEven;
}
Then you can rewrite the if-else statement so it calls the IsEven method to determine
whether number is even:
if (IsEven (number) )
{
MessageBox.Show("The number is even.");
}
else
{
MessageBox.Show("The number is odd.");
}
```

Example 8

Pay and Bonus application

In this application, enter an employee's gross pay and bonus amount and calculates the amount of retirement contribution.

In addition to regular pay, a company pays its employees an annual bonus. The company also contributes 5 percent of an employee's total compensation (gross pay plus bonus) to a retirement account.



Example 8- Code

```
public partial class Form1 : Form
{
// Constant field for the contribution rate
private const decimal CONTRIB_RATE = 0.05m;
public Form1()
{
InitializeComponent();
}
// The InputIsValid method converts the user input and stores
// it in the arguments (passed by reference). If the conversion
// is successful, the method returns true.Otherwise it returns
// false.
private bool InputIsValid(ref decimal pay, ref decimal bonus)
{
// Flag variable to indicate whether the input is good
bool inputGood = false;
// Try to convert both inputs to decimal.
if (decimal.TryParse(grossPayTextBox.Text, out pay) )
{
if (decimal.TryParse(bonusTextBox.Text, out bonus) )
{
// Both inputs are good.
inputGood = true;
}
else
{
// Display an error message for the bonus.
MessageBox.Show("Bonus amount is invalid.");
}
}
else
{
// Display an error message for gross pay.
MessageBox.Show("Gross pay is invalid.");
}
// Return the result.
return inputGood;
}
```

Example 8- Code

```
private void calculateButton_Click(object sender, EventArgs e)
{
// Variables for gross pay, bonus, and contributions
decimal grossPay = 0m, bonus = 0m, contributions = 0m;
if (InputlsValid(ref grossPay, ref bonus))
{
// Calculate the amount of contribution.
contributions = (grossPay + bonus) * CONTRIB_RATE;
// Display the contribution.
contributionLabel.Text = contributions.ToString("c');
}
```


THANK YOU